

INSTITUTO FEDERAL DE ALAGOAS
CAMPUS PALMEIRA DOS ÍNDIOS
CURSO TÉCNICO INTEGRADO AO ENSINO MÉDIO EM INFORMÁTICA - 513

JOÃO VITOR VIEIRA DA COSTA

GUSTAVO PIANCÓ DOS SANTOS

PO-WEB: PESQUISA DE VALIDAÇÃO

PALMEIRA DOS ÍNDIOS, AL
2026

JOÃO VITOR VIEIRA DA COSTA

GUSTAVO PIANCÓ DOS SANTOS

PO-WEB: PESQUISA DE VALIDAÇÃO

Trabalho de redação realizado na disciplina de POWeb.
Professor: Carlos Jean Costa Cabral

PALMEIRA DOS ÍNDIOS, AL
2026

1. Introdução

A validação de dados em formulários web é uma das práticas mais importantes no desenvolvimento de aplicações modernas. Ela assegura que as informações fornecidas pelos usuários sejam corretas, completas e seguras antes de qualquer processamento ou armazenamento.

Este trabalho aborda três temas centrais da validação de formulários: a diferença entre validação no lado do cliente (**Client-Side**) e no lado do servidor (**Server-Side**); os atributos nativos de validação do HTML5 (required, pattern, minlength e maxlength); e o uso das pseudo-classes CSS :valid e :invalid para fornecer feedback visual imediato.

---

2. Validação Client-Side vs. Server-Side

2.1 Validação no Lado do Cliente (Client-Side)

A validação **Client-Side** é executada diretamente no navegador do usuário, antes do formulário ser enviado ao servidor. As principais tecnologias envolvidas são os atributos HTML5 de validação e JavaScript, utilizando a Constraint Validation API nativa do navegador.

Vantagens:

- Feedback imediato para o usuário, melhorando significativamente a experiência (UX);
- Reduz o tráfego de rede ao evitar envios de formulários inválidos;
- Permite validações dinâmicas e complexas em tempo real.

Desvantagens:

Não oferece segurança real. A validação pode ser facilmente burlada desabilitando JavaScript, editando o HTML via DevTools ou enviando requisições diretamente com cURL/Postman. Por isso, nunca deve ser a única camada de validação.

2.2 Validação no Lado do Servidor (Server-Side)

A validação **Server-Side** ocorre no backend após o recebimento dos dados. É realizada com linguagens e frameworks como PHP, Python

(Django/Flask), Node.js, Java, etc., ou com bibliotecas especializadas de validação.

Vantagens:

- Segurança essencial: é a única forma confiável de garantir integridade dos dados;
- Protege contra ataques como injeção SQL, XSS e manipulação de dados;
- Princípio fundamental: **"Never trust user input"** (Nunca confie na entrada do usuário).

Desvantagens:

Feedback não é instantâneo (requer roundtrip com o servidor) e pode aumentar ligeiramente a carga caso muitos formulários inválidos sejam enviados.

2.3 Discussão e Recomendação

A melhor prática é utilizar **ambas as abordagens de forma complementar**. Use **Client-Side** para proporcionar excelente UX e reduzir tráfego desnecessário. Use **Server-Side** como camada obrigatória de segurança. Um sistema bem projetado implementa as duas.

---

3. Atributos de Validação Nativa no HTML5

O HTML5 introduziu atributos que permitem validação básica de forma nativa, sem JavaScript para casos simples.

3.1 Atributo required

Torna o campo obrigatório. O navegador impede o envio se o campo estiver vazio.

Exemplo:

HTML

```
<form action="/cadastro" method="post">
    <label for="nome">Nome completo</label>
    <input type="text" id="nome" name="nome" required>

    <label for="email">E-mail</label>
    <input type="email" id="email" name="email" required>

    <button type="submit">Cadastrar</button>
```

```
</form>
```

3.2 Atributo pattern (Expressões Regulares)

Permite validar o formato usando Expressão Regular. O valor deve corresponder exatamente ao padrão.

Exemplo de CEP brasileiro (aceita 00000-000 ou 00000000):

HTML
```
<label for="cep">CEP</label>
<input type="text" id="cep" name="cep"
       pattern="\d{5}-?\d{3}"
       placeholder="00000-000" required>
```

3.3 Atributos minlength e maxlength

Controlam o número mínimo e máximo de caracteres permitidos.

Exemplo de campo de senha:

HTML
```
<label for="senha">Senha</label>
<input type="password" id="senha" name="senha"
       minlength="8" maxlength="20" required
       placeholder="Digite uma senha de 8 a 20 caracteres">
```

---

4. Estados Visuais com Pseudo-classes CSS :valid e :invalid

As pseudo-classes CSS :valid e :invalid permitem aplicar estilos automaticamente de acordo com o estado de validade do campo.

4.1 Funcionamento

- **:valid** — Aplicada quando o campo atende todas as restrições (obrigatório preenchido, formato correto, tamanho adequado).
- **:invalid** — Aplicada quando o campo não atende aos critérios de validação.

Essas classes são atualizadas em tempo real conforme o usuário digita.

4.2 Exemplos Práticos de Estilização

CSS

```css
/* Estilo base */
input, textarea, select {
    padding: 10px;
    border: 2px solid #ccc;
    border-radius: 6px;
    font-size: 16px;
    transition: all 0.3s ease;
}

/* Campo VÁLIDO */
input:valid, textarea:valid, select:valid {
    border-color: #28a745;
    background-color: #f0fff0;
}

/* Campo INVÁLIDO */
input:invalid, textarea:invalid, select:invalid {
    border-color: #dc3545;
    background-color: #fff5f5;
}

/* Efeito ao focar */
input:focus:valid {
    box-shadow: 0 0 0 4px rgba(40, 167, 69, 0.2);
    outline: none;
}

input:focus:invalid {
    box-shadow: 0 0 0 4px rgba(220, 53, 69, 0.2);
    outline: none;
}
```

4.3 Boas Práticas e Acessibilidade

- Não dependa apenas da cor (importante para acessibilidade — daltônicos);
- Mantenha labels claros e mensagens de erro textuais;
- Use transições suaves para evitar mudanças bruscas;
- Teste em diferentes navegadores (Chrome, Firefox, Edge, Safari).

---

5. Conclusão

Este trabalho apresentou os principais conceitos e técnicas de validação de formulários web. A validação **Client-Side** é excelente para feedback rápido e UX, mas não oferece segurança. A validação **Server-Side** é obrigatória para garantir integridade e proteção contra ataques.

Os atributos nativos do HTML5 (required, pattern, minlength e maxlength) permitem validação básica simples. As pseudo-classes CSS :valid e :invalid oferecem feedback visual imediato e elegante. Quando utilizadas em conjunto, essas técnicas resultam em formulários robustos, seguros, acessíveis e com excelente usabilidade.

---

6. Referências

- Mozilla Developer Network (MDN) — Validação de formulários no lado do cliente. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn/Forms/Form_validation
- Mozilla Developer Network (MDN) — Constraint Validation API. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/Constraint_validation
- WHATWG — HTML Living Standard: Forms.
- OWASP — Input Validation Cheat Sheet.
- W3Schools — HTML5 Form Validation.